

RAPPORT DE STAGE D'ÉTÉ

Présenté en vue de l'obtention de la

LICENCE EN ELECTOTECHNIQUE , ELECTRONIQUE ET AUTOMATIQUE

Parcours : Informatique Industrielle

Par

KHAOULA TRABELSI

Réécriture des outils du soft de test de Sagemcom depuis langage shell vers python

Réalisé au sein de Tunisie Sagemcom mégrine



Encadrant professionnel : Mahmoud KALLEL

Année Universitaire : 2023-2024

Dédicaces

Je dédie ce travail à tous mes chers et ma profonde gratitude :

A mon cher père

Tous les mots du monde ne peuvent exprimer les innombrables sacrifices que vous avez consentis et le soutien que vous m'avez apporté tout au long de mes études. De l'amour et de l'affection dont tu m'as toujours entouré durant ces années d'études.

A ma belle MAMAN

Qui a été à mes côtés tout au long de mes études . Merci pour tout le soutien et l'amour que vous m'avez apporté depuis mon enfance. J'espère que vous trouverez dans cet ouvrage un véritable témoignage de mon profond amour et de ma gratitude éternelle.

A mes chères Soeurs

Je vous remercie pour vos encouragements et vos soutiens. Je vous aime de tout mon profond coeur.

A tous ceux qui m'aiment.

A tous ceux que j'aime.

Remerciement

Je tiens à exprimer mon vif remerciement et ma sincère gratitude à mon encadrant de **SAGEMCOM** Monsieur **Mahmoud KALLEL** qui est gardé un oeil attentif sur le déroulement du projet en donnant des remarques constructives et pour son soutien tout au long du stage .

Aussi, mes profondes remerciements et grâces totales s'adressent à Monsieur **Paul-Erwan RIO** et à Monsieur **Eric Lebihan** pour leurs disponibilités et leurs judicieux conseils. Je suis honoré par leurs commentaires.

Nous exprimons aussi notre gratitude à toute **l'équipe BSP** d'avoir pris le temps de partager leurs connaissances et leurs expertises spécialement Monsieur **Rayen GHRIBI** pour leur aide et leur conseils.

De plus, je tiens à exprimer mes sincères remerciements au directeur général de SST Monsieur **JANNOT Christian** et au directeur du département BBS Monsieur **BEN SALAH Naoufel**.

Et enfin ,Je voudrais exprimer ma gratitude et remerciements à ma chère famille et tous ceux qui m'a donné leur soutien.

Abréviation

API :Application Programming Interface

BSP : Board Support Package

CLI : Command-Line Interface

IP : Internet Protocol

LED : light Emitting Diode

Mac :Media Access Control

Manuf os : Manufacturing Operating System

OS :Operating System

PC :Personnal Computer

PP :Permanent Parameter

RAM : Random Access Memory

Scos : Sagemcom Operating System

SSD :Solid State Drive

TFTP : Trivial File Transfer Protocol

USB : Universal Serial Bus

WiFi : Wireless Fidelity

Table des matières

Remerciement	2
Introduction Générale	8
1 Présentation du cadre de projet	9
1.1 Introduction	9
1.2 Présentation de l'organisme d'accueil	9
1.2.1 Présentation du Groupe Sagemcom	9
1.2.2 Sagemcom software & technologies	9
1.2.3 Équipe BSP	10
1.2.4 L'organigramme de SST	11
1.3 Présentation du projet	11
1.3.1 Problématique	11
1.3.2 Etude de l'existant	11
1.3.3 Solution proposée	12
1.4 Planning prévisionnel	13
1.5 Conclusion	13
2 Etude De L'Art	14
2.1 Introduction	14
2.2 Présentation de software	14
2.2.1 Architecture Linux embarquée :	14
2.2.2 Linux embarqué	15
2.2.3 Manufacturing OS	16
2.2.4 scos-manuf-tools	16
2.3 Présentation de matériel	16
2.4 Implémentation en extender	16
2.4.1 Extender	16
2.4.2 Séquence de démarrage de l'Extender	17
2.5 Protocol TFTP	18
2.5.1 Définition :	18
2.5.2 Les avantages et les inconvénients du TFTP	19
2.5.3 La fonctionnalité de protocole TFTP :	19
2.6 Mémoire Flash	20
2.6.1 Image NAND	20
2.6.2 Image PKGTB	21
2.6.3 Difference entre Image NAND et Image PKGTB	21
2.7 Conclusion	21

3	Conception	22
3.1	Introduction	22
3.2	Méthodologie de conception	22
3.3	Le processus de développement	22
3.4	Analyse de besoin	23
3.4.1	Identification des besoins :	23
3.5	Analyse et Conception	24
3.5.1	Diagramme de séquence	24
3.6	Conclusion	24
4	Réalisation	25
4.1	Introduction	25
4.2	Présentation de l'environnement Software et Hardware	25
4.2.1	Environnement software :	25
4.2.2	Environnement hardware :	26
4.3	Installation de l'environnement de projet	26
4.3.1	Installation de git	26
4.3.2	Installation de l'environnement de travail	27
4.3.3	scos-manuf-tools	28
4.4	Communication utilisation-extender	29
4.4.1	Compilation de script	29
4.4.2	Configuration de l'extender	29
4.5	Protocole TFTP	31
4.6	Test et Conversion de script de shell vers python	35
4.6.1	Comment convertir des scripts shell en Python	35
4.6.2	Gérer le code avec Gitlab	39
4.6.3	Défis liés à la conversion de scripts shell en Python :	40
4.7	Conclusion	40
	Conclusion Générale	41
	Bibliographie	42

Table des figures

1.1	L'organigramme de SST	11
1.2	Comparaison entre python et c++	12
1.3	Planning préévisonnel	13
2.1	Architecture de linux embarqué	14
2.2	Séquence de démarrage de linux	17
2.3	Caractéristique de TFTP	19
3.1	Processus de développement	23
3.2	Diagramme de séquence	24
4.1	update le paquet apt	26
4.2	installation git	26
4.3	version de git	27
4.4	configuration Git	27
4.5	Configuration	27
4.6	Récupération des sources	27
4.7	Construction	28
4.8	clonning scos-manuf-tools	28
4.9	les scripts de scos-manuf-tools	28
4.10	Rebuild	29
4.11	manuf	29
4.12	Copier	29
4.13	minicom	30
4.14	Stockage de l'image dans la carte	30
4.15	Efface de la flash de la carte	30
4.16	Ecrire l'image dans la carte	31
4.17	Reset	31
4.18	Mise à jour de l'image pkgb	31
4.19	update le package apt	31
4.20	Installation le service tftpd-hpa	32
4.21	Vérification la fonctionnement de tftpd-hpa	32
4.22	Ouvre le fichier tftpd-hpa	32
4.23	Modification le fichier tftpd-hpa	32
4.24	Enregistrement de la modification de fichier tftpd-hpa	33
4.25	Création un nouveau répertoire /tftp	33
4.26	Remplacement les propriété de /tftp	33
4.27	Redémarrage de service tftpd-hpa	33
4.28	Vérification la fonctionnement de service tftpd-hpa	33

4.29	update le package apt	34
4.30	Installation tftpd-hpa	34
4.31	Trouve l'adresse ip	34
4.32	Active le mode verbose	34
4.33	Teste le fichier verif.sh a partir du répertoire de travail actuel	34
4.34	Teste le fichier verif.sh a partir du serveur tftp et quitter le tftp	35
4.35	Teste de script	35
4.36	test de scos-ipconfig	36
4.37	test de scos-mac	36
4.38	test de scos-wifi	36
4.39	test de scos-leds	36
4.40	test de scos-leds	37
4.41	test de scos-version	37
4.42	test de version de scos-ipconfig	37
4.43	Installer pylint avec sudo	38
4.44	Installer pylint avec pip	38
4.45	Script avant la correction	38
4.46	Script après la correction	39
4.47	Git branch	39
4.48	Git checkout	39
4.49	Git commit	39
4.50	Git push	40

Introduction Générale

Sagemcom est un fournisseur leader dans le domaine des équipements de communication, notamment dans les domaines des communications numériques, des réseaux, des télécommunications et des solutions pour la maison connectée. Ils fabriquent une gamme d'appareils, y compris des modems, des routeurs, des décodeurs TV, des compteurs intelligents, et d'autres dispositifs de communication et de connectivité.

En effet, certaines entreprises utilisent le script pour lancer ces commandes de fabrication qui sont incluses dans le logiciel de fabrication du domaine de système embarqué même si le script peut être complexe et contient plus de limites.

Alors que SAGEMCOM a évolué et planifié de réécrire les outils scos manuf en utilisant python qui offre plusieurs fonctionnalités. Cette évolution était un moyen efficace pour la gestion de mémoire, la sécurité et la performance.

C'est dans ce contexte qui s'inscrit notre projet intitulé "Réécriture des outils du soft de test de Sagemcom depuis langage shell vers python" effectué au sein de la société SAGEMCOM.

Ce rapport est constitué de quatre chapitres organisés comme suit :

Chapitre1 : Présentation du cadre de projet

Ce chapitre comprend une présentation générale de la société ainsi qu'une description du contexte général du projet.

Chapitre2 : Etude De L'Art

Ce chapitre contient une vue théorique de software et materiel .Aussi une présentation de protocol TFTP .

Chapitre3 : Conception

Ce chapitre analyse les détails des scénarios proposé dans ce projet.

Chapitre4 : Réalisation

Ce chapitre explique les différentes étapes de l'installations des outil de soft de test de projet .Ainsi, il explique la méthode de test et de conversion des scripts .

Chapitre 1

Présentation du cadre de projet

1.1 Introduction

Nous allons introduire dans ce premier chapitre le cadre du projet, à savoir l'organisme d'accueil. Par la suite, nous passerons à la présentation du sujet, illustré par les critiques de l'existence et les solutions que nous avons proposées.

1.2 Présentation de l'organisme d'accueil

1.2.1 Présentation du Groupe Sagemcom

Sagemcom est un groupe industriel français, leader mondial des produits et solutions de communication, des solutions audio et vidéo, et de l'énergie (électricité, gaz, eau) pour le marché du haut débit. Sagem conçoit, fabrique et livre chaque année plus de 40 millions de terminaux dans le monde.

Cette entreprise industrielle multinationale est composée d'unités de recherche et développement ainsi que d'usines qui sont de nouvelles technologies (décodeurs, impression et multimédia, etc.)

1.2.2 Sagemcom software & technologies

Sagemcom Software et Technologies est la filiale de recherche et de développement en Tunisie du groupe Sagemcom qui compte parmi les leaders européens sur le marché des terminaux communicants à haute valeur ajoutée. La filiale R&D a été implantée en Tunisie en 2005 et compte aujourd'hui plus de 600 ingénieurs. Aujourd'hui, les principaux métiers de R&D sont représentés à SS&T : développement et qualification mécanique, électronique, Software embarqué, Software serveur.

Pôle AVS : le rôle de pôle AVS est le développement, la validation et l'intégration des logiciels embarqués pour les décodeurs des télévisions numériques en diffusion Broadcast et IP.

Famille de produits :

- Développement et la validation des logiciels/ produits issus de Middleware .
- SAGEM linux /os21,Android
- déclinaisons et variantes pour opérateurs .

- Gamme de produits : zapper SD/HD , 4K , PVR HD.

Pole E&T : le role de pole E&T est le developpement , la validation et l'integration de produits smart city .

Famille de produits :

- Smart Meter .
- Smart Grid .
- Data concentrator .
- Internet of Things .

Pole BBS : le role de pole BBS est le developpement , la validation et l'integration des logiciels embarques pour les passerelles residentielles et screens .

Famille de produits :

- Passerelle Residentielle .
- Home Screen.Outils Swib .

1.2.3 Équipe BSP

L'équipe BSP (Board Support Package), en français "équipe de support de la carte", est une équipe de développeurs ou d'ingénieurs logiciels spécialisés dans le développement et la maintenance de logiciels embarqués, notamment pour les systèmes embarqués sur une carte matérielle spécifique. Le BSP, ou package de support de carte, est un ensemble de logiciels, de pilotes, de configurations et d'outils nécessaires pour faire fonctionner et prendre en charge efficacement le matériel d'une carte ou d'un système embarqué particulier.

La principale responsabilité de l'équipe BSP est de s'assurer que le système d'exploitation ou le logiciel embarqué peut fonctionner correctement sur une plateforme matérielle donnée.

Cela implique généralement les tâches suivantes :

Activité :

- Adaptation du système d'exploitation .
- Développement de pilotes de périphériques .
- Personnalisation du bootloader .
- Configuration et tests .
- Maintenance et mises à jour

1.2.4 L'organigramme de SST

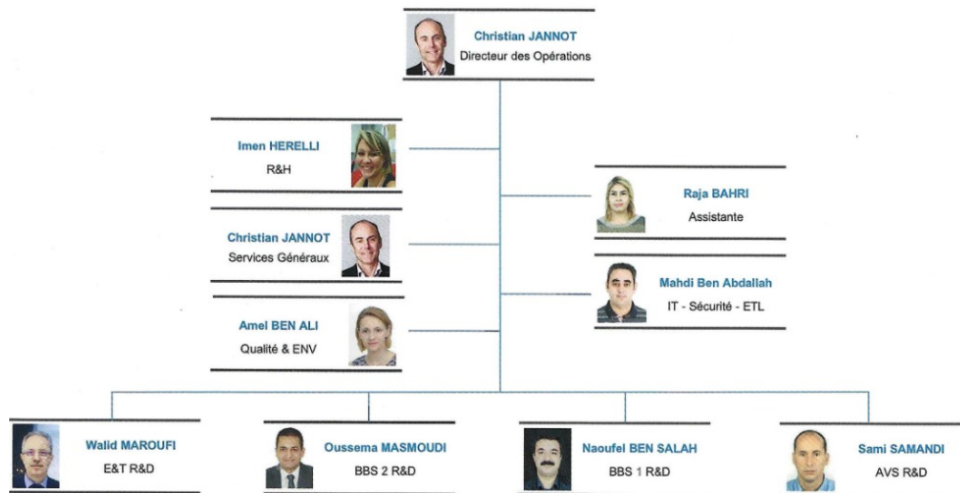


FIGURE 1.1 – L'organigramme de SST

1.3 Présentation du projet

Notre projet intitulé "scos-manuf-tools" fait partie de notre projet de stage d'été à l'institut supérieur des technologies de l'information et de la communication (ISTIC). Il s'agit d'un stage obligatoire aux deuxième années. Le stage a débuté le 6 juillet 2023 chez Sagemcom software & technologies.

1.3.1 Problématique

SAGEMCOM utilise le script Shell comme un outil de développement qui :

- Limites de performances : les scripts Shell ne sont pas aussi performants que les langages de programmation ce qui peut poser un problème si vous avez des exigences de performances sérieuses.
- Utiliser plus de RAM.
- Complexité : Il peut être difficile de gérer les scripts Shell en raison de la syntaxe limitée et du manque de structure. (Accéder au moins de fonction).
- Sécurité : Entraîne un problème de sécurité si l'entrée n'est pas correctement validée (cela peut conduire à des vulnérabilités d'injection de commande).

L'idée principale est de réécrire les outils du soft de test de sagemcom avec une meilleure sécurité et performance.

⇒ l'objectif c'est l'amélioration.

D'où, il faut transférer de script shell vers un autre langage qui permet de satisfaire nos besoins.

1.3.2 Etude de l'existant

Parmi les langages existants, nous retrouvons :

1. Python : Python est largement utilisé dans les technologies modernes, comme l'intelligence artificielle et l'apprentissage automatique. Python convient à la création

de modèle d'apprentissage automatique en raison de sa prise en charge de diverses bibliothèques.

- Python est un langage de programmation orienté objet de haut niveau (C'est un langage plus faciles et plus simples à apprendre, car proche du pseudocode).
 - Prend en charge La modèles de programmation orientée objet (Il contient également des fonctionnalités telles que la liaison dynamique et la saisie dynamique).
 - Offre une structure et une prise en charge idéales pour les grands programmes.
 - Python offre une prise en charge du garbage collection automatique.
 - Il prend en charge un mode interactif de test et de débogage.
2. Servir pour créer des systèmes embarqués. Il est plus proche du matériel et est donc préféré pour la fabrication de gadgets tels que des montres intelligentes, des instruments médicaux, des appareils IoT.
- Le C++ est considéré comme un langage de programmation de niveau intermédiaire, car il contient à la fois les fonctionnalités d'une programmation de haut niveau et d'un langage de programmation de bas niveau.
 - Le code doit suivre l'architecture spécifique et doit être portable.
 - Le code doit être facilement " interprété et dynamique "

1.3.3 Solution proposée

Fonctionnalité	C++	PYTHON
Compilation	Un langage compilé	Un langage interprété
Utilisation	Pas facile contrairement à Python en raison du syntaxe complexe	Plus facile d'écrire du code
Vitesse d'exécution	Plus rapide	Lente
Nature de langage	Statique	Dynamique
Portabilité	N'est pas portable	Portable
Gestion de la mémoire	Ne prend pas en charge le Garbage collector	Prend en charge le Garbage collector
Efficacité	Moins propre et moins maniable que Python	Plus facile à maintenir, orienté objet et plus simple à utiliser
Porté des variables	La portée des variables est limitée aux boucles.	Les variables sont accessibles même en dehors de la boucle
Prototypage rapide	Le prototypage rapide n'est pas possible en raison d'une plus grande taille de code	Le prototypage rapide est possible grâce à la petite taille du code

FIGURE 1.2 – Comparaison entre python et c++

Après une étude comparative des différentes solutions existantes, il était nécessaire de proposer une solution qui répondrait à nos besoins au vu des lacunes constatées. Nous avons choisi de travailler avec PYTHON qui permet de manipuler des fichiers, l'analyse

des données et coder des script courts et simple pour profiter de plusieurs avantages au niveau de la gestion de mémoire qui nous rend utiliser moins de RAM.

Pour atteindre nos objectifs, nous avons suivi la démarche suivante :

1. Générer le soft de test SAGEMCOM.
2. Concevoir l'architecture et développer les outils du soft de test.
3. Tester ces outils sur les cartes de SAGEMCOM.

1.4 Planning prévisionnel

Pour résumer et mettre en perspective les activités et réalisations du stage, j'ai créé un planning . Le document détaillé ci-dessus est destiné à fournir un aperçu des tâches sur lesquelles j'ai travaillé et des objectifs que j'ai atteints.

Nom de la tâche ▼	Durée ▼	Début ▼	Fin ▼
Résolution des problèmes en shell et python	1,63 jours	Jeu 06/07/23	Ven 07/07/23
Installation	2,63 jours	Lun 10/07/23	Mer 12/07/23
Documentation sur python	0,75 jour	Jeu 13/07/23	Jeu 13/07/23
Installation TFTP et flashage de l'image NAND	0,75 jour	Ven 14/07/23	Ven 14/07/23
Test et conversion des scripts	35 jours	Lun 17/07/23	Ven 08/09/23

FIGURE 1.3 – Planning prévisionnel

1.5 Conclusion

Dans ce premier chapitre, nous avons commencé par présenter notre organisme d'accueil SAGEMCOM. Ensuite, nous avons décrit le contexte du stage, puis nous avons dressé la problématique de notre projet, tout en fournissant les critiques et les solutions envisageables.

Après avoir terminé la présentation du premier chapitre, nous pouvons ainsi entamer la prochaine étape ETUDE DE L'ART.

Chapitre 2

Etude De L'Art

2.1 Introduction

Après avoir donné un aperçu sur le contexte de notre projet, nous allons effectuer dans ce deuxième chapitre un balayage sur les principales notions théoriques que nous avons utilisées durant la réalisation de ce projet, Tout d'abord, nous allons commencer par une présentation de la Software et Matériel .Ensuite, nous allons définir le protocole . Enfin, donnerons des définitions des termes techniques utilisés dans le projet.

2.2 Présentation de software

2.2.1 Architecture Linux embarquée :

Au niveau le plus élémentaire, un système Linux embarqué est un système qui utilise Linux comme système d'exploitation situé entre le matériel et l'application d'un périphérique embarqué. Il existe cinq composants clés dans un système Linux embarqué :

1. Processeur matériel
2. Couche d'abstraction matérielle
3. Système d'exploitation Linux
4. Couche de service
5. Couche d'application

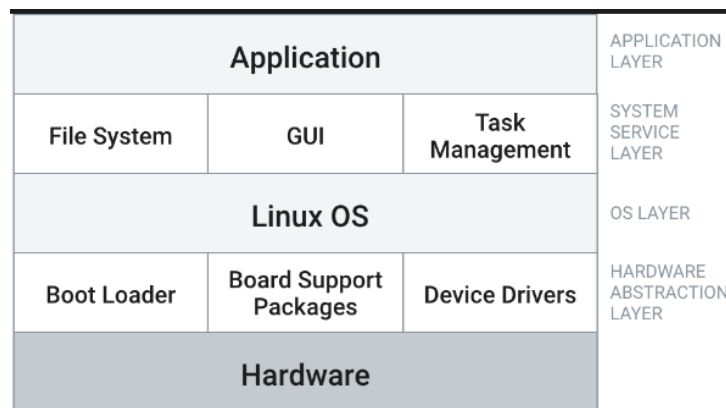


FIGURE 2.1 – Architecture de linux embarqué

2.2.2 Linux embarqué

Linux embarqué est une version allégée du système d'exploitation Linux conçue pour être utilisée dans des appareils embarqués tels que des systèmes électroniques, des objets en réseau et des équipements industriels. Cette technologie permet aux développeurs de créer des applications et des systèmes personnalisés pour répondre à des besoins spécifiques. Il existe plusieurs outils disponibles pour le développement sur Linux embarqué, tels que Buildroot et les chaînes d'outils de compilation croisée. Ces outils sont utilisés pour créer des systèmes Linux embarqués personnalisés.

C'est quoi Buildroot ?

Buildroot est un outil open source permettant de :

- Créer des systèmes Linux embarqués personnalisés.
- Fournit une infrastructure pour créer des systèmes embarqués à partir de zéro en utilisant une approche basée sur les composants. Il est conçu pour être simple, flexible et facile à utiliser, même pour les personnes ayant peu d'expérience en développement embarqué.
- Sélectionner les packages logiciels dont vous avez besoin pour votre système et de les combiner pour créer une image de système de fichiers racine personnalisée.

Scos-devel : Il s'agit de l'environnement utilisé pour générer des versions de logiciels de test. Généralement utilisé par BSPTEAM pour implémenter /tester diverses fonctionnalités et correctifs. Il est également utilisé dans divers projets pour tester certaines fonctionnalités sans intervention d'un middleware.

C'est quoi Cross-compilation toolchain ?

Cross-compilation toolchain est un ensemble d'outils logiciels qui :

- Permet de compiler le code source sur une plateforme autre que celle sur laquelle il s'exécute.
- Est extrêmement utile car les plates-formes embarquées disposent souvent de ressources limitées en termes de puissance de traitement et de stockage.
- Permet aux développeurs de compiler leur code sur un ordinateur et le déployer sur des systèmes embarqués tels que Raspberry Pi ou des systèmes basés sur ARM.

L'outil make : Les outils Make sont des programmes qui peuvent automatiser la compilation de logiciels. Vous utilisez un fichier appelé Makefile pour décrire les dépendances entre les différents modules de votre projet. Ces fichiers permettent à make de déterminer quelles parties de votre code doivent être recompilées et dans quel ordre. L'objectif principal de l'outil Make est de faciliter le processus de développement logiciel en automatisant la compilation. Cela permet aux développeurs de se concentrer sur l'écriture de code au lieu de gérer les dépendances et les compilations. De plus, make vous permet de modifier des parties de votre code sans avoir à recompiler l'intégralité du projet, ce qui vous permet d'économiser du temps et des ressources.

Comment utiliser le Buildroot avec Cross-compilation toolchain

L'utilisation de Buildroot avec vos outils de compilation croisée présente de nombreux avantages.

- Il vous permet de développer des systèmes Linux embarqués personnalisés pour une variété de plates-formes cibles sans avoir à gérer plusieurs outils de compilation différents.
- Les outils de compilation croisée peuvent réduire considérablement le temps de compilation et minimiser les erreurs de compilation.
- Vous pouvez utiliser Buildroot avec cross-compilation toolchain pour personnaliser facilement les composants du système Linux embarqué. Ceci est particulièrement utile pour les projets nécessitant des fonctionnalités spécifiques.

2.2.3 Manufacturing OS

Manufacturing est le système d'exploitation chargé de fournir tous les outils permettant de tester les appareils sur la chaîne de montage. Il est basé sur le noyau Linux et le système de fichiers racine et utilise l'environnement de construction scos-devel. Le package le plus important du système de fichiers racine est scos-manuf-tools. Il fournit toutes les commandes permettant de configurer et de tester divers composants et fonctionnalités de l'appareil (DSL, paramètres permanents, activation de la chaîne de confiance, etc.).

2.2.4 scos-manuf-tools

Le module scos-manuf-tools fournit tous les outils utilisés lors du processus de fabrication pour :

1. Tester les composants matériels (LED, écrans, etc).
2. Tester les propriétés de l'appareil (par exemple WLAN, DSL, PON, Ethernet, etc).
3. Fournit des informations d'étalonnage (WiFi, PON) à l'appareil.
4. Mettre à jour l'appareil avec un système d'exploitation client/terrain.

2.3 Présentation de matériel

Dans ce projet, nous utilisons des appareils WiFi extenders et des modems qui sont basés sur différentes propriétés matérielles et technologies.

- Connectivité : DSL, PON, DOCSIS, LTE/5G, Ethernet, MoCa/HPNA, WiFi, VoIP, USB, Bluetooth, ZigBee, Thread, NFC.
- Stockage : NAND, eMMC.
- Sécurité : stockage secret, génération de nombres aléatoires, accélérateurs cryptographiques, chaîne de confiance, chaîne de chiffrement, environnement d'exécution de confiance...
- Capteurs : température, tension, courant, pression, proximité, etc.
- Interface utilisateur : LED, boutons, écrans...

2.4 Implémentation en extender

2.4.1 Extender

Un extender est un appareil qui étend la portée des signaux électroniques tels que le WiFi ou le Bluetooth. Il est souvent utilisé dans les grandes maisons et les bureaux

pour étendre la portée des réseaux sans fil. Les extensions peuvent considérablement améliorer l'expérience de communication entre votre PC et d'autres appareils. En fait, les prolongateurs vous permettent d'étendre la portée de votre signal WiFi, vous permettant ainsi de connecter des appareils éloignés de votre routeur sans sacrifier la qualité de la connexion.

2.4.2 Séquence de démarrage de l'Extender

Le bootloader, le kernel et l'utilisateur sont des éléments importants du système d'exploitation d'un ordinateur. La communication entre eux est essentielle pour garantir un fonctionnement efficace et stable de l'ensemble du système. Sans une compréhension claire de ces communications, il peut être difficile de diagnostiquer et de résoudre les problèmes qui surviennent lors de l'utilisation de votre ordinateur. De plus, une compréhension approfondie des communications kernel-utilisateur peut vous aider à optimiser les performances du système et à développer des applications plus efficaces.

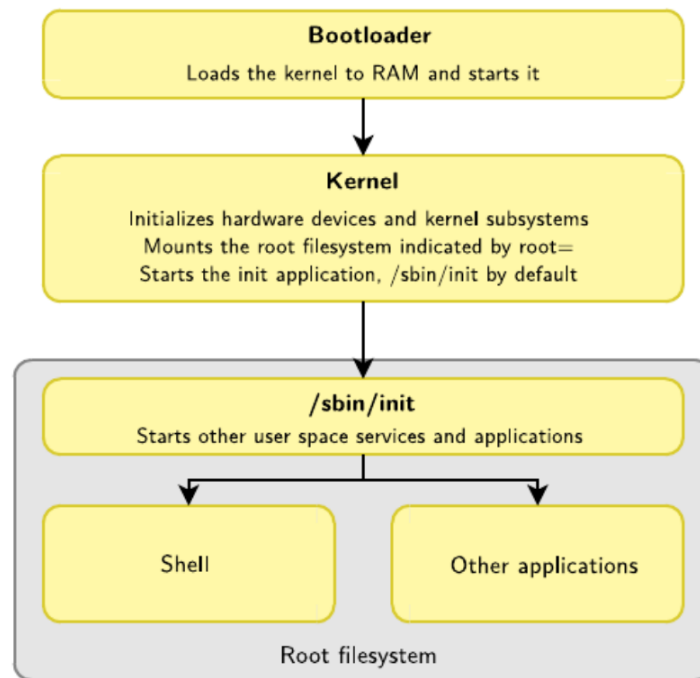


FIGURE 2.2 – Séquence de démarrage de linux

Le rôle du bootloader : Le bootloader est un programme qui s'exécute avant le système d'exploitation principal et dont le rôle consiste à charger le système d'exploitation en mémoire. Il permet également la communication entre le matériel informatique et le système d'exploitation. En fait, il initialise le matériel et fournit les informations nécessaires au système d'exploitation pour interagir de manière optimale avec le matériel.

Le rôle du kernel : Le kernel est le cœur du système d'exploitation. Responsable de la gestion des ressources matérielles et logicielles telles que la mémoire, les périphériques

d'entrée/sortie et les processus. Il est également responsable de la sécurité du système et gérer les autorisations et l'accès aux ressources. Il joue un rôle important dans la communication entre le bootloader et l'utilisateur

La communication entre bootloader et kernel

la communication entre bootloader et kernel est essentielle au démarrage réussi du système d'exploitation. Cette communication s'effectue via des structures de données, appelées tables de configuration. Le bootloader remplit ces tableaux avec des informations sur le matériel de l'ordinateur, telles que la quantité de mémoire disponible et les appareils connectés. Le noyau utilise ensuite ces informations pour configurer le système et initialiser les pilotes nécessaires au bon fonctionnement de l'ordinateur.

La communication entre kernel et Le programme utilisateur(user)

Le kernel est le coeur du système d'exploitation et gère les ressources matérielles et logicielles de l'ordinateur. Communiquez avec les programmes utilisateur via des appels système. Les appels système sont des fonctions prédéfinies qui permettent aux programmes utilisateur d'accéder aux fonctionnalités du kernel. Vous pouvez utiliser des appels système pour effectuer des opérations telles que la lecture ou l'écriture de fichiers, la création de processus et la gestion de la mémoire. Les programmes utilisateur communiquent avec le kernel via des interfaces de programmation d'applications (API).

⇒ Au démarrage de système, le bootloader charge le kernel en mémoire et lui donne le contrôle. Le kernel initialise ensuite les composants matériels et logiciels nécessaires pour permettre la communication avec l'utilisateur. Le kernel est également responsable de la gestion des interruptions du système, permettant au système d'interagir avec des périphériques externes tels que le clavier et la souris.

2.5 Protocol TFTP

2.5.1 Définition :

C'est quoi un protocole TFTP ?

Le TFTP :

- Trivial File Transfer Protocol
- Est un protocole de transfert de fichiers simplifié conçu pour les situations auxquelles la simplicité et la rapidité sont plus importantes que la sécurité et les fonctionnalités avancées.
- Il fonctionne également selon le modèle client-serveur.
- Utilise le protocole UDP (User Datagram Protocol) au lieu de TCP ce qui le rend moins fiable mais plus rapide que le FTP. Permet d'effectuer un transfert de données sans connexion fixe entre les partenaires de communication.
- Pour les requêtes, le protocole TFTP utilise le port 69.

En résumé :

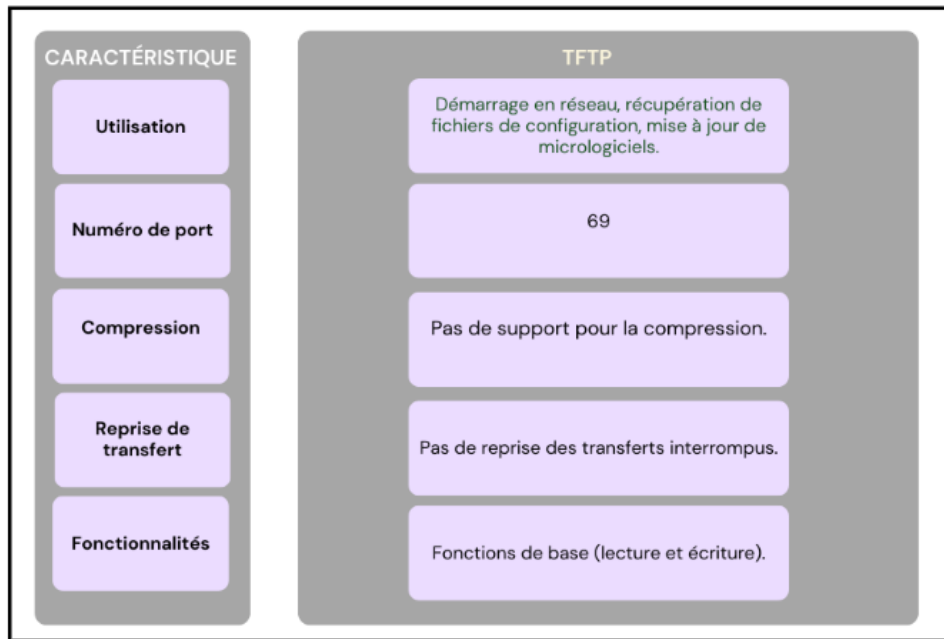


FIGURE 2.3 – Caractéristique de TFTP

2.5.2 Les avantages et les inconvénients du TFTP

Les avantages du TFTP

- Simple.
- Facile à implémenter.
- Favorise la transfert rapide des données.
- Economie de ressource : nécessite moins de ressource pour fonctionner Ce qui on fait un bon choix pour les systèmes embarqués.
- Maintenance légère : TFTP est utile pour la mise à jour de micrologiciels (firmware) sur des dispositifs réseau tels que les routeurs, les commutateurs et les points d'accès Wi-Fi car il ne nécessite pas une infrastructure complexe et fournit simplement le transfert de fichiers.

Les inconvénients du TFTP :

- **Sécurité limités** : TFTP ne fournit pas de mécanismes de sécurité avancés comme le chiffrement. Les données sont transférées en clair ce qui le rend vulnérable aux attaques de capture de paquets.
- **Manque de fonctionnalités avancées** : En raison de sa simplicité, TFTP ne prend pas en charge des fonctionnalités telles que la navigation dans les répertoires, la gestion des autorisations et la résiliation des transferts interrompus.
- **Absence de mécanismes de contrôle d'erreur avancés** : TFTP utilise des mécanismes de contrôle d'erreur basiques ce qui peut conduire à des problèmes de fiabilité sur des réseaux peu fiables.

2.5.3 La fonctionnalité de protocole TFTP :

Le processus de transfert d'un fichier comprend trois phases principales. En termes très généralisés, ce sont :

- **Connexion initiale** : le client TFTP établit la connexion en envoyant une demande initiale au serveur. Le serveur répond au client et la connexion est effectivement ouverte.
- **Transfert de données** : une fois la connexion établie, le client et le serveur échangent des messages TFTP. Un appareil envoie des données et l'autre envoie des accusés de réception.
- **Terminaison de la connexion** : lorsque le dernier message TFTP contenant des données a été envoyé et accusé de réception la connexion est terminée.

Le transfert de données par TFTP est toujours basé sur une requête-client qui demande un accès en écriture ou en lecture. Cette requête fait également office de demande de connexion qui est automatiquement établie si le serveur accepte l'accès.

Ensuite, le client ou le serveur transmet le fichier sous forme de blocs dont la taille est prédéfinie. Dans les premières versions du protocole on avait encore la valeur fixe de 512 octets mais depuis le RFC 2348 le serveur et le client peuvent ajuster la taille du bloc avec plus de souplesse.

Le transfert se fait bloc par bloc.

Tout bloc réceptionné doit être validé par un accusé de réception "Acknowledgement " avant que le paquet suivant ne puisse être envoyé.

Un paquet de données dont la taille est inférieure au nombre d'octets défini sera identifié comme paquet final du transfert.

- Si un paquet se perd pendant le transfert cela déclenche une fin d'attente chez le destinataire avec une nouvelle tentative de renvoi du dernier paquet. De cette manière, l'émetteur du paquet perdu est averti du fait qu'il doit renvoyer ce paquet.

2.6 Mémoire Flash

La mémoire flash est un périphérique de stockage informatique capable de stocker des données pendant de courtes périodes. Contrairement aux disques durs traditionnels, la mémoire flash peut conserver les données même après la mise hors tension. En tant que tel, il est idéal pour les appareils portables tels que les smartphones et les tablettes. La mémoire flash est également généralement plus fiable et plus rapide que les disques durs traditionnels. C'est pour cette raison qu'il devient de plus en plus populaire dans le monde informatique.

2.6.1 Image NAND

La mémoire flash NAND est un type de mémoire non volatile couramment utilisée dans divers appareils électroniques tels que les clés USB, les cartes mémoire, les disques SSD (SSD), les smartphones et de nombreux autres appareils. Et utilise une architecture série, dans laquelle les cellules mémoire sont connectées les unes après les autres.

Caractéristiques de la mémoire flash NAND

Mémoire non volatile : Ces mémoires peuvent être stockées pendant de longues périodes sans endommager les données.

Reprogrammable : Une puce électronique programmable peut stocker toutes sortes de données, des documents aux vidéos, afin qu'elles soient disponibles à tout moment.

Temps d'écriture et d'effacement plus rapide : cette mémoire est utilisée dans les applications où les écritures persistantes sont prioritaires sur la récupération des données.

Plus de stockage : La mémoire flash NAND est si dense que vous pouvez même acheter des périphériques de stockage de 2 To dans votre magasin informatique local.

Faible coût de fabrication

2.6.2 Image PKGTB

La mémoire flash PKGTB est un type de mémoire flash qui utilise des grilles de transistor flottantes pour stocker des données. la mémoire PKGTB stocke les données en parallèle, ce qui la rend plus rapide et plus efficace. En raison de sa vitesse et de son efficacité, la mémoire flash PKGTB est largement utilisée dans les applications nécessitant des performances élevées, telles que les disques SSD et les cartes mémoire hautes performances pour appareils photo et caméscopes.

2.6.3 Différence entre Image NAND et Image PKGTB

Les principales différences entre la mémoire flash NAND et le PKGTB c'est que :

1. La mémoire flash pkgtb est un binaire qui contient le soft (uboot + kernel + filesystem) et l'image nand contient tous les binaires loader (SPL+TPL+uboot+kernel+FS).
2. La mémoire flash NAND utilise une architecture en lignes, dans laquelle les cellules mémoire sont connectées les unes après les autres. En revanche, la mémoire flash PKGTB utilise une architecture parallèle, avec des cellules mémoire connectées en parallèle.
3. La mémoire flash PKGTB est généralement plus rapide à lire et à écrire que la mémoire flash NAND.
4. La mémoire flash NAND est plus fiable et a une durée de vie plus longue que la mémoire flash PKGTB.
5. La mémoire flash NAND est généralement moins chère que la mémoire flash PKGTB.

2.7 Conclusion

Tout au long de ce chapitre nous avons présenté les termes utilisés dans notre projet théoriquement.

Dans ce chapitre suivant nous allons traiter les méthodologies de conception .

Chapitre 3

Conception

3.1 Introduction

Après avoir décrit le contexte de notre projet, ce chapitre entame la phase de conception qui représente la méthodologie d'analyse qui prend en charge le déroulement de projet. Alors, on représente les principaux acteurs et leurs rôles. Ensuite dans la branche fonctionnelle, on va définir les besoins fonctionnels et non fonctionnels, puis présenter les différents diagrammes d'UML.

3.2 Méthodologie de conception

Nous réalisons ce travail en utilisant des concepts de langage appelés UML (Unified Modeling Language) est un langage de modélisation graphique basé sur des pictogrammes conçu comme une méthode de visualisation standardisée dans le domaine du développement logiciel et de la conception orientée objet. J'ai eu l'opportunité.

UML est basé sur une sémantique précise et une notation graphique expressive. Il définit les concepts de base et fournit également des mécanismes pour étendre ces concepts.

3.3 Le processus de développement

Le cycle de développement en cascade

Le modèle en cascade est un cycle de vie de projet qui nécessite une conception préliminaire en raison de contraintes matérielles élevées et des coûts élevés associés aux modifications de conception en cours.

Un modèle en cascade comprend les phases et résultats suivants :

- Test en shell : le script est testé pour comprendre son utilité .
- Conversion : le script est convertir de shell a python .
- Compilation : le script est compilé et copier sous TFTP .
- Test en python : le script est testé et vérifié qu'il fonctionne correctement.

Ci-dessous, le modèle du cycle de vie en cascade :

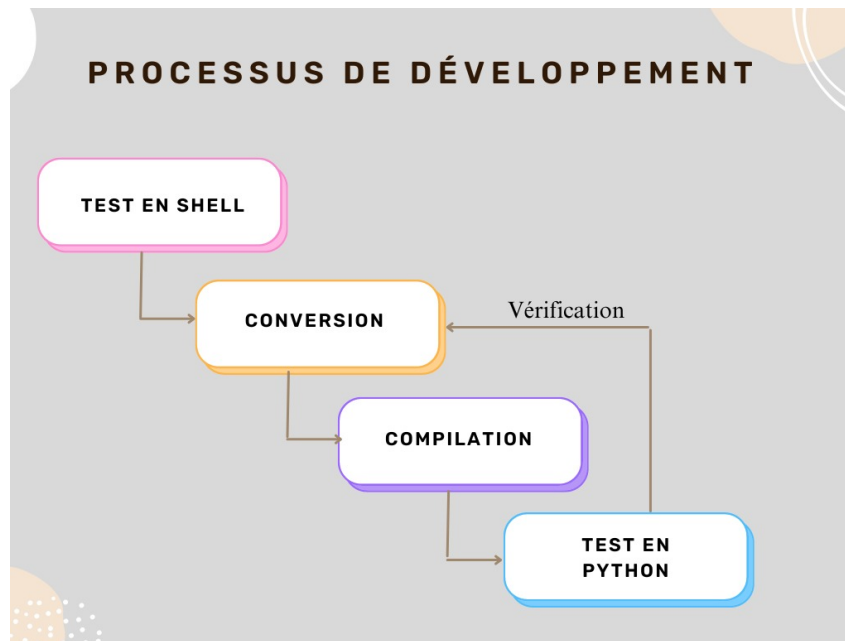


FIGURE 3.1 – Processus de développement

3.4 Analyse de besoin

3.4.1 Identification des besoins :

On présenter les fonctionnalités demandées de notre projet tout en spécifiant les besoins fonctionnels et non fonctionnels.

Besoins Fonctionnels :

Dans cette étape, nous expliquerons ce que nous attendons de notre application et modéliserons à l'aide du langage de modélisation UML.

Administrateur :

- Gestion de matériels.
- Gestion de logiciels.
- *jira*
- *gitlab*
- Gestion de réseau.

Développeur :

- Convertir le script de Shell vers python.
- Compiler le script.
- Tester.

Besoins non fonctionnels :

Une fois les besoins fonctionnels satisfaits, des besoins opérationnels doivent être pris en compte tout au long de développement.

Les besoins non fonctionnels présentent les exigences internes pour le système

Nous distinguons :

- Sécurité : garantir que les données sensibles (code source) sont protégées contre toute forme d'accès non autorisé, de manipulation et de vol.
- Gestion de la mémoire : évite les fuites et les débordements de mémoire pouvant entraîner des pannes et des ralentissements inattendus.
- Performance : fait référence à l'efficacité et à la réactivité d'un système qui garantit que le système peut accomplir les tâches dans un délai acceptable même sous une charge importante.

3.5 Analyse et Conception

Nous avons atteint une étape critique du processus. Grâce aux recherches menées dans la partie précédente . c'est elle qui met l'accent sur le rôle de chaque acteur du système et les fonctions .

3.5.1 Diagramme de séquence

Le diagramme de séquence permet de représenter l'interaction entre les objets ou composants dans un système logiciel. Il met en évidence la manière dont les différents éléments du système interagissent les uns avec les autres au fil du temps.

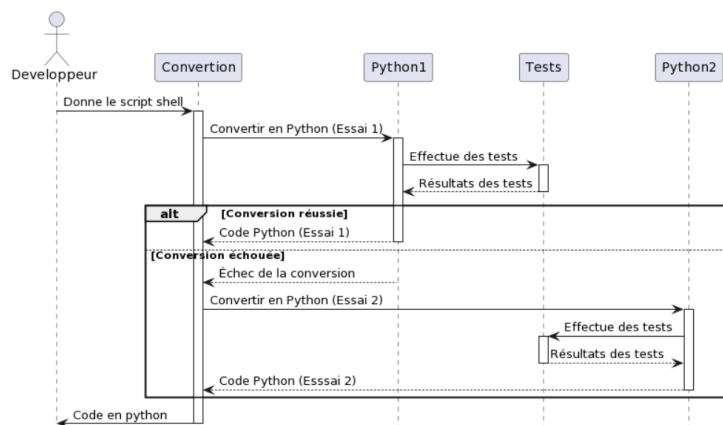


FIGURE 3.2 – Diagramme de séquence

3.6 Conclusion

Dans ce chapitre , nous avons décrit la démarche de notre projet étape par étape pour être plus claire. En outre , nous avons détaillé les fonctionnalités et les processus de développement à utiliser .

Dans le chapitre suivant, l'idée générale c'était de présenter le déroulement de travail côté installation , conversion , compilation et test .

Chapitre 4

Réalisation

4.1 Introduction

Après avoir achevé la partie théorique et la partie de conception, nous allons entamer l'étape de la réalisation qui représente une partie très importante du temps consacré à ce projet.

Au cours de ce chapitre, nous allons aborder l'environnement Software et Hardware. avec lequel nous avons pu réaliser notre projet.

Ensuite, nous allons présenter les captures d'installations des outils nécessaires

Enfin, nous allons intégrer les captures de tester en shell et en python et des exemples des conversions des scripts ainsi que les merges requête

4.2 Présentation de l'environnement Software et Hardware

4.2.1 Environnement software :

afin d'assurer la réussite du développement de notre projet , nous avons eu besoin de diverses ressources logicielles.

- Linux
- TFTP server
- Minicom
- Jira
- Gitlab

Jira :

JIRA est une plateforme multifonction qui vise à faciliter la gestion de projet en aidant à suivre les tâches, identifier les blocages et partager des informations entre les membres d'une équipe. À l'origine dédiée au développement logiciel, la solution peut aujourd'hui s'adapter à des projets marketing, finance, RH, etc.

Gitlab :

GitLab est une plateforme de collaboration qui étend les fonctionnalités de Git, un système de gestion de code source.

GitLab fournit une interface Web avec les fonctionnalités suivantes :

- Gérer les droits d'accès par rôle (manager, développeur, rapporteur).
- Navigation dans le code source.
- Le code source est soumis via un mécanisme de "merge request".
- Revue des changements de code source.

Bien entendu, GitLab vous permet de procéder à des validations croisées avec JIRA et de fusionner les demandes et les problèmes.

4.2.2 Environnement hardware :

Afin d'assurer la fonctionnalité du développement de notre projet , nous avons eu besoin de diverses ressources materiel.

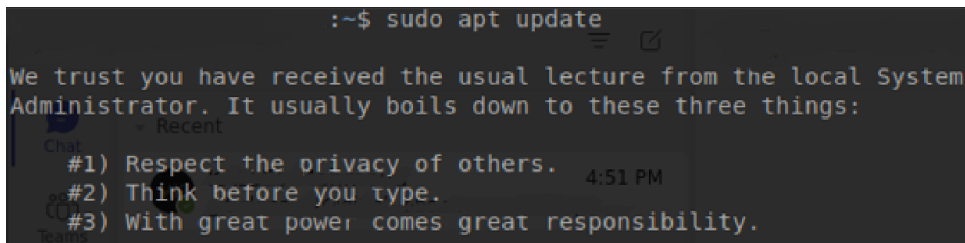
- PC basée sur Linux
- un extender sagemcom utilisant le type de stockage nand et prenant en charge les fichiers logiciels pkgtb

4.3 Installation de l'environnement de projet

4.3.1 Installation de git

Les systèmes de contrôle de version comme Git sont essentiels aux meilleures pratiques modernes de développement de logiciels. Le contrôle de version vous permet de suivre les logiciels au niveau source. Vous pouvez suivre les modifications, revenir aux étapes précédentes et créer des versions alternatives de fichiers et de répertoires.

Tout d'abord, mettez à jour votre index de paquet local à l'aide de l'outil de gestion de paquet apt.



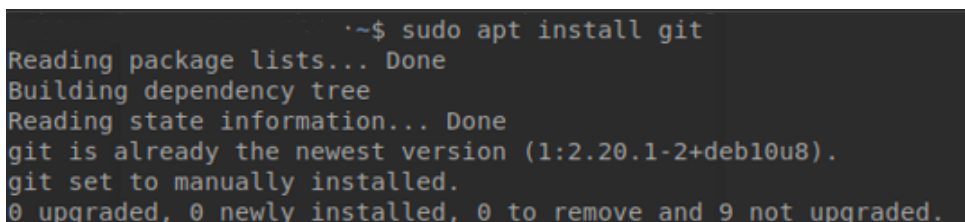
```

:~$ sudo apt update
We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:
#1) Respect the privacy of others.
#2) Think before you type.
#3) With great power comes great responsibility.

```

FIGURE 4.1 – update le paquet apt

Une fois la mise à jour terminée, vous pouvez installer Git.



```

:~$ sudo apt install git
Reading package lists... Done
Building dependency tree
Reading state information... Done
git is already the newest version (1:2.20.1-2+deb10u8).
git set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 9 not upgraded.

```

FIGURE 4.2 – installation git

Vous pouvez vérifier que Git a été correctement installé en exécutant la commande suivante et en voyant si vous obtenez le bon résultat.

```

:~$ git --version
git version 2.20.1

```

FIGURE 4.3 – version de git

Les informations que vous saisissez sont enregistrées dans un fichier de configuration Git. En option, vous pouvez le modifier manuellement à l'aide d'un éditeur de texte tel que le suivant (nous utilisons geany) :

```

:~$ touch ~/.gitconfig
:~$ geany ~/.gitconfig

```

FIGURE 4.4 – configuration Git

NB : les information saisir dans gitconfig sont des données personnel .

4.3.2 Installation de l'environnement de travail

Récupération de scos-devel :

Pour installer scos-devel ,une copie locale du code doit être téléchargée..

Pour créer une image, vous devez suivre ces étapes :

1. Configuration :

```

:~$ cd scos-devel
~/scos-devel$ make O=$HOME/build/scos-devel/sc f266v2lg/manuf sc f266v2lg manuf defconfig
make -C /home/g701769/scos-devel/buildroot BR2_EXTERNAL=/home/g701769/scos-devel/scos/base:/home/g701769/scos-devel/scos/airoha
scos-devel/scos/foss:/home/g701769/scos-devel/scos/irdeto:/home/g701769/scos-devel/scos/intel:/home/g701769/scos-devel/scos/mxl
scos-devel/scos/quectel:/home/g701769/scos-devel/scos/qcom:/home/g701769/scos-devel/scos/silabs:/home/g701769/scos-devel/scos/sw

```

FIGURE 4.5 – Configuration

2. Récupération des sources :

```

~/scos-devel$ make O=$HOME/build/scos-devel/sc f266v2lg/manuf source
make -C /home/g701769/scos-devel/buildroot BR2_EXTERNAL=/home/g701769/scos-devel/scos/base:/home/g701769/scos-devel/scos/airoha
scos-devel/scos/foss:/home/g701769/scos-devel/scos/irdeto:/home/g701769/scos-devel/scos/intel:/home/g701769/scos-devel/scos/mxl
scos-devel/scos/quectel:/home/g701769/scos-devel/scos/qcom:/home/g701769/scos-devel/scos/silabs:/home/g701769/scos-devel/scos/sw
scos-devel/scos/voice:/home/g701769/scos-devel/scos/wifi BR2_DL_DIR=/home/g701769/sources/br2 source

```

FIGURE 4.6 – Récupération des sources

3. Construction :

```
~/scos-devel$ make O=$HOME/build/scos-devel/sc_f266v2lg/manuf
make -C /home/g701769/scos-devel/buildroot BR2_EXTERNAL=/home/g701769/scos-devel/scos/bas
scos-devel/scos/foss:/home/g701769/scos-devel/scos/irdeto:/home/g701769/scos-devel/scos/i
cos-devel/scos/quectel:/home/g701769/scos-devel/scos/qcom:/home/g701769/scos-devel/scos/s
```

FIGURE 4.7 – Construction

Conclusion :

A la fin de la construction, l'image est générée sous le chemin suivant .

\$ HOME/build/scos-devel/sc_f266v2lg/manuf/images

4.3.3 scos-manuf-tools

Installation scos-manuf-tools

Pour installer scos-manuf-tools ,une copie de code original doit être téléchargée depuis gitlab.

```
~/scos-devel$ git clone git@gitlab:scos-manuf-tools.git
Cloning into 'scos-manuf-tools'...
remote: Enumerating objects: 1207, done.
remote: Counting objects: 100% (1207/1207), done.
remote: Compressing objects: 100% (523/523), done.
remote: Total 8016 (delta 782), reused 1083 (delta 684), pack-reused 6809
Receiving objects: 100% (8016/8016), 1.44 MiB | 5.49 MiB/s, done.
Resolving deltas: 100% (5153/5153), done.
```

FIGURE 4.8 – cloning scos-manuf-tools

Les scripts de Module scos-manuf-tools

```
~/scos-manuf-tools$ ls scripts/
cc-curpower-dump.in      scos-finalize.in      scos-modem-quectel.in  scos-secrets-itstore.in  scos-version-boot-cfe-ram.in
Makefile.am             scos-garrison-brcm.in  scos-modem-rg500.in    scos-sensors-hwmon.in    scos-version-boot-cfe-rom.in
pon-calibration          scos-garrison.in      scos-modem-rg520.in    scos-sensors.in          scos-version-boot-pattern.in
scos-accel.in            scos-gpio-brcmcheap.in  scos-modem-telit.in    scos-sfp-brcm.in         scos-version-boot-qcom-sbl.in
scos-accel-asa301.in     scos-gpio.in           scos-nfe.in             scos-sfp.in              scos-version-boot-qcom-spf.in
scos-audio.in            scos-gpio-ms51.in      scos-notice.in          scos-sfp-intel.in        scos-version-boot-qcom-tz.in
scos-ble.in              scos-gpio-sysfs.in     scos-notice-leds.in     scos-srom-eeeprom.in     scos-version-boot-smc.in
scos-brinks.in           scos-ipconfig.in       scos-notice-screen.in  scos-srom.in             scos-version-boot-uboot.in
scos-cert-check-qcom.in  scos-keys-generic.in   scos-omci-brcm.in      scos-srom-nvram.in       scos-version-brcm-bootfs.in
scos-devlock-brcm-gen3-brmk.in  scos-keys.in          scos-omci.in           scos-storage-hwmon.in    scos-version-firmware-ble.in
scos-devlock-brcm-gen3-optee-brmk.in  scos-laser-brcm.in    scos-otp.in            scos-storage.in          scos-version-firmware-dct.in
scos-devlock-brcm-gen3-turnkey-68880.in  scos-laser.in         scos-pon.in            scos-storage-sdcard.in   scos-version-firmware-dhd.in
scos-devlock-brcm-gen3-turnkey.in        scos-laser-macom.in   scos-postman.in        scos-storage-usb.in      scos-version-firmware-mcu.in
scos-devlock-brcm-gen3-turnkey-sgc.in    scos-leds-brcm-netled.in  scos-ppp.in           scos-storage-wifi-brcm-eeeprom.in  scos-version-firmware-qtn.in
scos-devlock.in              scos-leds-generic.in   scos-runner.in         scos-thermal-hwmon.in    scos-version-firmware-quectel.in
scos-devlock-qcom-ipq807x.in  scos-leds.in           scos-sbox.in           scos-thermal.in          scos-version-firmware-rg500.in
scos-devprov.in             scos-leds-ledhandler.in  scos-sbox-process-nupp.in  scos-thermal-ms51.in    scos-version-firmware-rg520.in
scos-dhcp.in                scos-mac.in            scos-sbox-process-pp-f3896lg.in  scos-thermal-qtn.in     scos-version-firmware-telit.in
scos-dsl-brcm.in            scos-manuf-data-file.in  scos-sbox-process-pp-f5685.in  scos-thermal-rg500.in   scos-version-firmware-wifi-qcom.in
scos-dsl.in                 scos-manuf-data.qtn.in  scos-sbox-process-pp.in  scos-thermal-rg520.in    scos-version-firmware-zb.in
scos-dsl-vrx.in             scos-manuf-data-wifi-brcm-eeeprom.in  scos-screen-fbdev.in    scos-thermal-telit.in   scos-version-gsdf.in
scos-fan-generic.in         scos-manuf-expect-qtn.in  scos-screen.in         scos-thermal-zone.in     scos-version.in
scos-fan.in                scos-modem.in           scos-secrets.in        scos-version-boot-bios.in  scos-version-misc-brcm.in
scos-file-sign-ub-preload.in  scos-modem.in           scos-secrets.in        scos-version-boot-bytel.in  scos-version-misc-intel.in
```

FIGURE 4.9 – les scripts de scos-manuf-tools

4.4 Communication utilisation-extender

4.4.1 Compilation de script

pour compiler le code source

Pour compiler un script, on utilise les commandes suivantes :

"Rebuild" est le processus de reconstruction à partir du code source. Ils apportent généralement des modifications au code source, et ces modifications doivent être prises en compte pour produire une nouvelle version exécutable du logiciel.

```

:~$ cd scos-devel/
:~/scos-devel$ make O=$HOME/build/scos-devel/sc_f266v2lg/manuf scos-manuf-tools-rebuild
make -C /home/g701769/scos-devel/buildroot BR2_EXTERNAL=/home/g701769/scos-devel/scos/base:/home/g701769/scos-devel/scos/airoha:/home/
scos-devel/scos/foss:/home/g701769/scos-devel/scos/irdeto:/home/g701769/scos-devel/scos/intel:/home/g701769/scos-devel/scos/mxl:/home/
scos-devel/scos/quectel:/home/g701769/scos-devel/scos/qcom:/home/g701769/scos-devel/scos/silabs:/home/g701769/scos-devel/scos/swan:/home/

```

FIGURE 4.10 – Rebuild

La compilation est le processus de conversion du code source écrit dans un langage de programmation en un langage binaire compréhensible par la machine.

```

:~/scos-devel$ make O=$HOME/build/scos-devel/sc_f266v2lg/manuf
make -C /home/g701769/scos-devel/buildroot BR2_EXTERNAL=/home/g701769/scos-devel/scos/base:/home/g701769/scos-devel/scos/airoha:/home/g701769/
scos-devel/scos/foss:/home/g701769/scos-devel/scos/irdeto:/home/g701769/scos-devel/scos/intel:/home/g701769/scos-devel/scos/mxl:/home/g701769/
scos-devel/scos/quectel:/home/g701769/scos-devel/scos/qcom:/home/g701769/scos-devel/scos/silabs:/home/g701769/scos-devel/scos/swan:/home/g7017
scos-devel/scos/voice:/home/g701769/scos-devel/scos/wifi BR2_DL_DIR=/home/g701769/sources/br2

```

FIGURE 4.11 – manuf

```

:~$ cp ~/build/scos-devel/sc_f266v2lg/manuf/images/sc_f266v2lg.scos.pkgtb /tftp

```

FIGURE 4.12 – Copier

4.4.2 Configuration de l'extender

Pour configurer l'extender, nous aurons besoin de quelques étapes essentielles :

- Connecté l'extender à une source d'alimentation.
- Insère l'extrémité du câble dans un des ports LAN de l'extender et l'autre extrémité dans un convertisseur USB Ethernet. Cet convertisseur se branche sur un port USB de PC.
- Branchez l'extrémité du câble CLI dans le port approprié de l'extender. Il devrait y avoir un port spécifié pour la connexion réseau et prenez l'autre extrémité du câble CLI et branchez-la sur le pc
- Ouvrir le terminal et lancer la commande dans la figure ci-dessous :

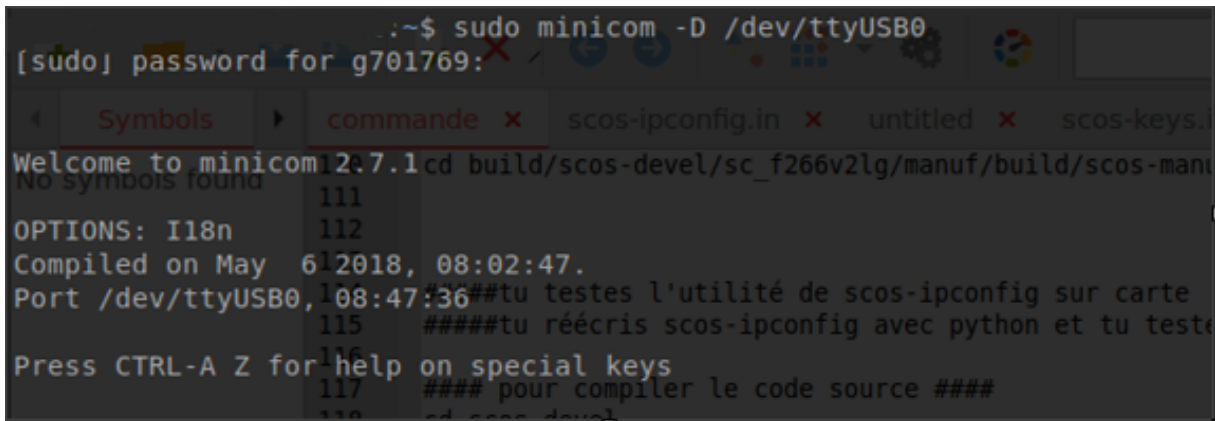


FIGURE 4.13 – minicom

Mise à jour de l'image nand

- Stockage de l'image dans la RAM de la carte à l'adresse 0x1290000.

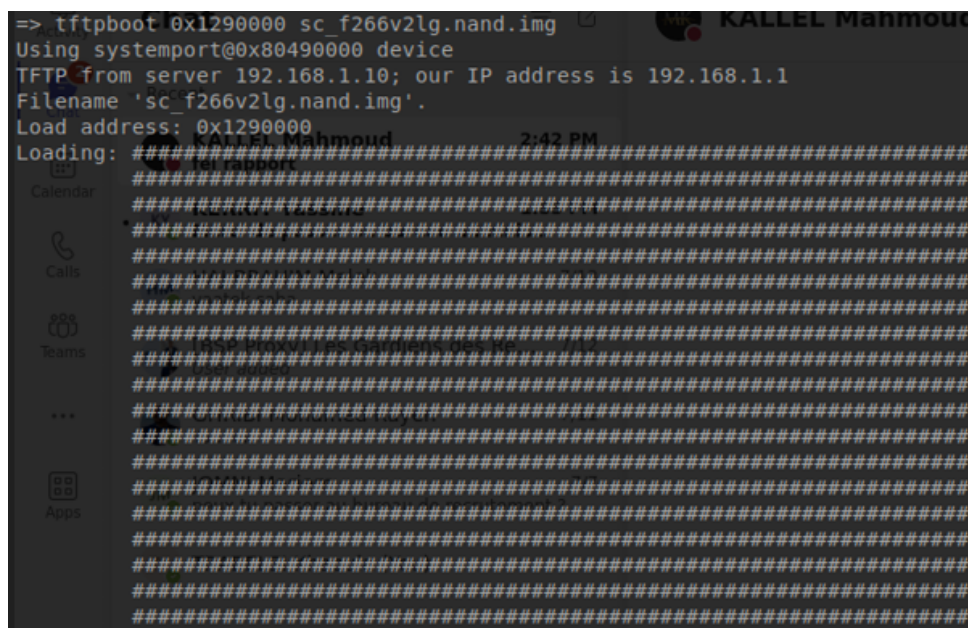


FIGURE 4.14 – Stockage de l'image dans la carte

- Effacer la flash de la carte



FIGURE 4.15 – Efface de la flash de la carte

- Ecrire l'image dans la carte


```
=> nand write 0x1290000 0 ${filesize}
```

```
=> |reset JM JOMNI Mariem 7/7  
resetting ... peux tu passer au bureau de recrutement ?  
-----  
BTRM TK TRABELSI Khaoula (You)
```

```
=> sdk flash img upgrade sc f266v2lg.scos.pkgtb
Using systemport@0x80490000 device
TFTP from server 192.168.1.10: our IP address is 192.168.1.1 id/scos-manuf-to
Filename 'sc_f266v2lg.scos.pkgtb'.
Load address: 0x1000000
Loading: #####
```

```
:~$ sudo apt update
```


Maintenant, nous installons le paquet tftpd-hpa avec la commande suivante :

```
~$ sudo apt install tftpd-hpa
Reading package lists... Done
Building dependency tree
Reading state information... Done
tftpd-hpa t:3 http://us.archive.ubuntu.com/ubuntu/
Hit:4 http://us.archive.ubuntu.com/ubuntu/
Fetched 88.7 kB in 1s (82.1 kB/s)
Reading package lists... Done
```

FIGURE 4.20 – Installation le service tftpd-hpa

Maintenant, vérifiez si le service tftpd-hpa fonctionne avec la commande suivante :

```
~$ sudo systemctl status tftpd-hpa
● tftpd-hpa.service - LSB: HPA's tftp server
   Loaded: loaded (/etc/init.d/tftpd-hpa; generated)
   Active: active (running) since Fri 2023-07-14 10:37:47 CET; 1min 17s ago
     Docs: man:systemd-sysv-generator(8)
    Tasks: 1 (limit: 4915)
```

FIGURE 4.21 – Vérification la fonctionnement de tftpd-hpa

Configuration du serveur TFTP : Le fichier de configuration par défaut du serveur tftpd-hpa est /etc/default/tftpd-hpa. Si vous configurez un serveur TFTP, vous devez modifier ce fichier de configuration puis redémarrer le service tftpd-hpa. Pour modifier le fichier de configuration /etc/default/tftpd-hpa, exécutez la commande suivante :

```
~$ sudo nano /etc/default/tftpd-hpa
```

FIGURE 4.22 – Ouvre le fichier tftpd-hpa

Vous devez ouvrir le fichier de configuration pour le modifier. Il s'agit de la configuration par défaut du serveur TFTP.

```
# /etc/default/tftpd-hpa
TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/var/lib/tftpboot"
TFTP_ADDRESS="0.0.0.0:69"
TFTP_OPTIONS="--secure "
```

FIGURE 4.23 – Modification le fichier tftpd-hpa

Le fichier de configuration final ressemblera à ceci : Appuyez sur <Ctrl> + x, puis y, puis pour enregistrer les modifications.

```
# /etc/default/tftpd-hpa

TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/tftp"
TFTP_ADDRESS="0.0.0.0:69"
TFTP_OPTIONS="--secure --create "
```

FIGURE 4.24 – Enregistrement de la modification de fichier tftpd-hpa

Nous devons maintenant créer un nouveau répertoire /tftp. Pour ce faire, exécutez la commande suivante :

```
~$ sudo mkdir /tftp
[sudo] password for g701769:
```

FIGURE 4.25 – Création un nouveau répertoire /tftp

Ensuite, remplacez le propriétaire et le groupe du répertoire /tftp par tftp à l'aide de la commande suivante :

```
~$ sudo chown tftp:tftp /tftp
[sudo] password for g701769:
```

FIGURE 4.26 – Remplacement les propriété de /tftp

Redémarrez ensuite le service tftpd-hpa à l'aide de la commande suivante :

```
~$ sudo systemctl restart tftpd-hpa
[sudo] password for g701769:
```

FIGURE 4.27 – Redémarrage de service tftpd-hpa

Maintenant, vérifiez si le service tftpd-hpa est en cours d'exécution à l'aide de la commande suivante :

```
~$ sudo systemctl status tftpd-hpa
● tftpd-hpa.service - LSB: HPA's tftp server
   Loaded: loaded (/etc/init.d/tftpd-hpa; generated)
   Active: active (running) since Wed 2023-09-06 12:42:50 CEST; 18s ago
     Systemd: Started
```

FIGURE 4.28 – Vérification la fonctionnement de service tftpd-hpa

Comme vous pouvez le constater, le service tftpd-hpa est en cours d'exécution. La configuration est donc réussie.

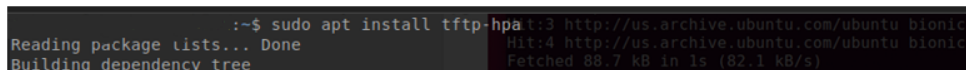
Test du serveur TFTP : Maintenant, pour accéder au serveur TFTP, vous avez besoin d'un programme client TFTP. Il existe de nombreux programmes clients TFTP. Vous n'aurez probablement pas besoin d'un autre que pour tester le serveur TFTP car les appareils qui utiliseront le serveur TFTP auront déjà le programme client installé dessus. Pour installer le client `tftp-hpa` TFTP sur Ubuntu, exécutez les commandes suivantes. Le



```
:~$ sudo apt update
```

FIGURE 4.29 – update le package apt

programme client `tftp-hpa` doit être installé. Maintenant, exécutez la commande suivante



```
:~$ sudo apt install tftp-hpa
Reading package lists... Done
Building dependency tree
Hit:3 http://us.archive.ubuntu.com/ubuntu bionic-
Hit:4 http://us.archive.ubuntu.com/ubuntu bionic-
Fetched 88.7 kB in 1s (82.1 kB/s)
```

FIGURE 4.30 – Installation `tftpd-hpa`

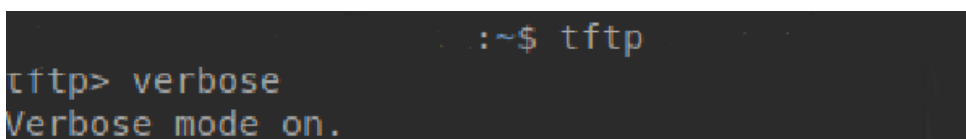
sur votre serveur TFTP pour trouver l'adresse IP du serveur TFTP.



```
:~$ ip a
```

FIGURE 4.31 – Trouve l'adresse ip

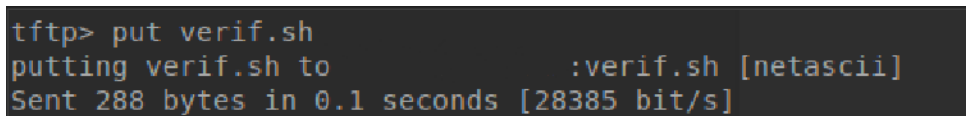
Maintenant, vous connectez au serveur TFTP et vous activez le mode verbose avec la commande suivante :



```
:~$ tftp
tftp> verbose
Verbose mode on.
```

FIGURE 4.32 – Active le mode verbose

Maintenant, pour télécharger un fichier (`verif.sh`) à partir du répertoire de travail actuel, exécutez la commande suivante :



```
tftp> put verif.sh
putting verif.sh to :verif.sh [netascii]
Sent 288 bytes in 0.1 seconds [28385 bit/s]
```

FIGURE 4.33 – Teste le fichier `verif.sh` à partir du répertoire de travail actuel

Pour télécharger le fichier (verif.sh) a partir du serveur tftp et quitter le shell tftp , exécuter les commandes suivantes :

```
tftp> get verif.sh
getting from          :verif.sh to verif.sh [netascii]
Received 288 bytes in 0.1 seconds [30573 bit/s]
tftp> quit
```

FIGURE 4.34 – Teste le fichier verif.sh a partir du serveur tftp et quitter le tftp

4.6 Test et Conversion de script de shell vers python

4.6.1 Comment convertir des scripts shell en Python

La conversion de scripts shell en Python peut sembler difficile, mais c'est en réalité très simple. La première étape consiste à tester et à comprendre la syntaxe de base de notre script shell existant et à identifier les parties qui doivent être converties en Python. Ensuite, une fois que nous avons identifié les parties du script qui doivent être converties, nous pouvons commencer à écrire le code Python. Cependant, afin d'automatiser le processus de conversion, il est important d'inspecter manuellement le code généré pour s'assurer qu'il fonctionne correctement.

Tester et comprendre le script shell :

On test les script shell avec les commandes trouver dans le " manufacturing book " *comment tester un script ?*

Après avoir redémarré la carte d'extension, pour constater les utilites de ce script, nous devons exécuter des commandes spécifiques au script telles que les suivantes :

```
# scos-keys -v 24 BACKENDS.append(BACKEND)
0.143.1 25 BACKENDS string = ' '.join(BACKENDS)
# scos-keys -h 26 print(BACKENDS string)
Usage: {} [OPTIONS]
Options:
  -h Show this help message
  -H Show in-depth manual of this command
  -v Show version information
  -b NAME Set NAME as backend to use, version information
Available backends:
Hints:
Use: 'scos-keys help' to get backend help message help' to get backend help m
```

FIGURE 4.35 – Teste de script

Quel est le but du script ?

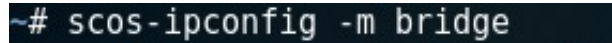
Chaque appareil possède ses propres caractéristiques (la plus importante étant le nombre de LED et de ports Ethernet). Cependant, scos-manuf-tools fournit une interface unique et générique qui résume tous les détails de l'appareil. Vous trouverez ci-dessous un exemple de propriétés du script du module scos-manuf-tools.

- **Scos-ipconfig :**

Utilise `scos-ipconfig` pour définir le mode d'adressage IP approprié à partir d'une liste de trois à quatre configurations prédéfinies.

- `default` : C'est le mode par défaut lors du démarrage.
- `bridge` : toutes les interfaces ont la plage d'adresses 192.168.1.1.
- `accessory` : toutes les interfaces ont une plage d'adresses 192.168.x.2.

Exemple :pour le configurer en mode bridge, exécuter :



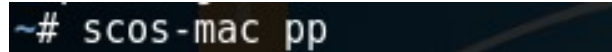
```
~# scos-ipconfig -m bridge
```

FIGURE 4.36 – test de `scos-ipconfig`

• **Scos-mac :**

Configure l'adresse MAC de l'interface à l'aide de `scos-mac`. Par défaut, toutes les interfaces se voient attribuer des adresses aléatoires.

Exemple :pour configurer une adresse Mac configurée avec des paramètres permanents, procédez comme suit :



```
~# scos-mac pp
```

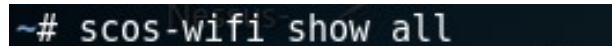
FIGURE 4.37 – test de `scos-mac`

• **Scos-wifi :**

Utilise `scos-wifi` pour contrôler l'interface WiFi. Une discussion courante est l'interface. nous pouvons nommer l'interface souhaitée à l'aide de la commande suivante :

- Broadcom (`wl0`, `wl1`, `eth0`,... .)
- Fréquence (2,4 GHz, 5 GHz)
- soit toutes les interfaces possibles (toutes)

Exemple :Pour obtenir des informations détaillées sur les interfaces disponibles, exécuter :



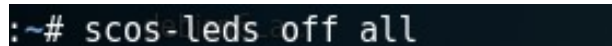
```
~# scos-wifi show all
```

FIGURE 4.38 – test de `scos-wifi`

• **Scos-leds :**

Utilise `scos-leds` pour contrôler les leds.

Exemple :Pour éteindre toutes les leds, exécuter :



```
:~# scos-leds off all
```

FIGURE 4.39 – test de `scos-leds`

Exemple :Les leds sont référencées par des noms, qui sont déclarés dans le dtb. Pour obtenir la liste des leds disponibles avec leur nom, exécuter :

A terminal window with a dark background. The prompt is '~#' followed by the command 'scos-leds list'.

FIGURE 4.40 – test de scos-leds

- **Scos-version :**

Exemple : exécuter :

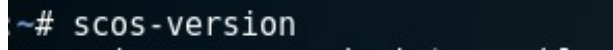
A terminal window with a dark background. The prompt is '~#' followed by the command 'scos-version'.

FIGURE 4.41 – test de scos-version

Exemple : Pour afficher la version d'une commande scos-* spécifique (par exemple scos-ipconfig), exécuter :

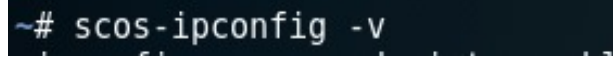
A terminal window with a dark background. The prompt is '~#' followed by the command 'scos-ipconfig -v'.

FIGURE 4.42 – test de version de scos-ipconfig

Conclusion : La généralité des commandes est assurée par le système backend, configurable via des fichiers.

Certaines fonctionnalités, telles que les mises à niveau du firmware, sont fournies via des projets supplémentaires tels que :scos-fw-upgrade.

Convertir le script de shell vers python :

1. **Familiariser avec Python** : Assure le bon comprendre de la syntaxe Python, les structures de contrôle (boucles, conditions), les opérations sur les fichiers, etc.
2. **Porter les commandes Shell vers Python** : Identifier les commandes shell (ls, grep, awk, sed, etc.) et trouvez leurs équivalents Python. Par exemple, pour lister les fichiers d'un répertoire de notre shell (ls), nous utilisons os.listdir() en Python.
3. **Gestion des fichiers** : Si le script travaille avec des fichiers dans des scripts shell, apprendre à effectuer ces opérations (open, read, write, etc.) en Python.
4. **Variables et chaînes de caractères** : Comprendre comment les variables sont déclarées et manipulées en Python. Noter que contrairement au shell, qui utilise principalement des guillemets simples, Python utilise des chaînes entourées de guillemets simples (") ou de guillemets doubles (")
5. **Fonctions** : Si le script shell contient des fonctions, convertir les fonctions en Python en définissant une fonction Python.

6. **Utilisation de modules spécifiques** : Si le script shell utilise des commandes ou des fonctions spécifiques qui ne sont pas disponibles directement dans Python, recherche un module ou une bibliothèque Python pouvant fournir les mêmes fonctions. `expl(subprocess ,os ,sys )`

Correction de script :

Pylint est un outil open source conçu pour aider les développeurs à examiner le code et sa qualité, à détecter les erreurs de programmation dans le langage Python et à fournir des suggestions simples de refactorisation.

Comment installer Pylint ?

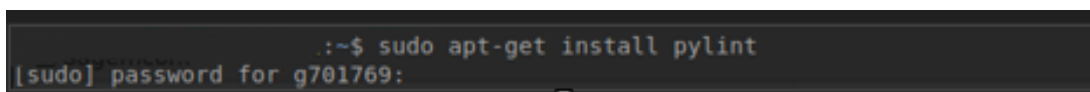


FIGURE 4.43 – Installer pylint avec sudo

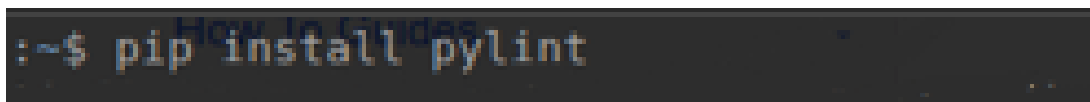


FIGURE 4.44 – Installer pylint avec pip

Comment utiliser le pylint ?

Une fois Pylint installé, on peut l'utiliser en exécutant la commande suivante depuis le répertoire du module dans Terminal :

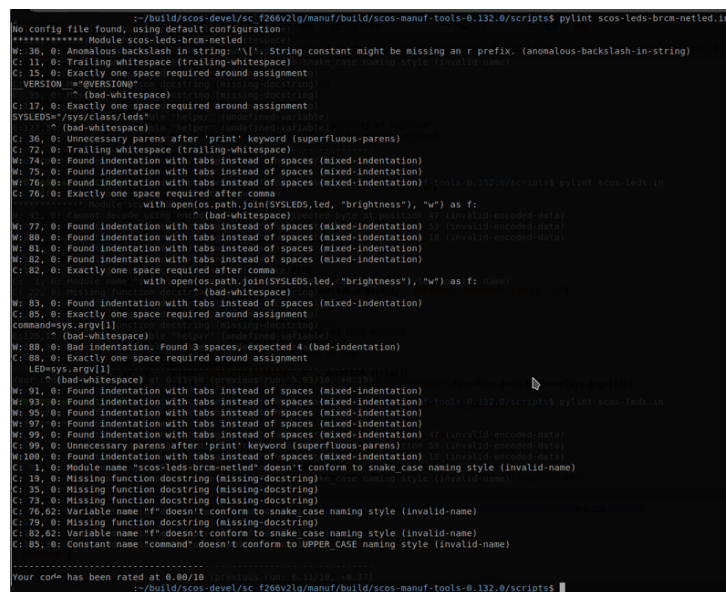


FIGURE 4.45 – Script avant la correction

Après la correction de Script :

```

~/build/scos-devel/sc_f266v2lg/manuf/build/scos-manuf-tools-0.132.0/scripts$ pylint scos-leds-brcm-netled.in
No config file found, using default configuration
***** Module scos-leds-brcm-netled
W: 36, 0: Anomalous backslash in string: '\['. String constant might be missing an r prefix. (anomalous-backslash-in-string)
C: 1, 0: Module name "scos-leds-brcm-netled" doesn't conform to snake_case naming style (invalid-name)
C: 19, 0: Missing function docstring (missing-docstring)
C: 35, 0: Missing function docstring (missing-docstring)
C: 73, 0: Missing function docstring (missing-docstring)
C: 76,72: Variable name "f" doesn't conform to snake_case naming style (invalid-name)
C: 79, 0: Missing function docstring (missing-docstring)
C: 82,72: Variable name "f" doesn't conform to snake_case naming style (invalid-name)
.....
Your code has been rated at 7.71/10 (previous run: 7.14/10, +0.57)

~/build/scos-devel/sc_f266v2lg/manuf/build/scos-manuf-tools-0.132.0/scripts$

```

FIGURE 4.46 – Script après la correction

4.6.2 Gérer le code avec Gitlab

Merge Request :

Pour faire une demande de fusion (merge request) avec Git, suivez ces étapes :

- *Git branch* : avant de commencer à travailler sur une fonctionnalité ou une correction de bug, créez une nouvelle branche à partir de la branche principale.

```
~/scos-manuf-tools$ git branch feature
```

FIGURE 4.47 – Git branch

- *Git checkout* : Elle permet de nous déplacer entre différentes branches, de restaurer des fichiers ou des commits, et bien plus encore

```
~/scos-manuf-tools$ git checkout feature
Switched to branch 'feature'
```

FIGURE 4.48 – Git checkout

- *Git commit* : Copier les nouveaux modifications sous le module. Une fois que vous avez terminé, faites des commits pour enregistrer nos changements.

```
~/scos-manuf-tools$ cp ~/build/scos-devel/sc_f266v2lg/manuf/build/scos-manuf-tools-0.132.0/scripts/scos-ipconfig.in scripts/
~/scos-manuf-tools$ git commit -m "scos-ipconfig: rewrite scos-ipconfig scripts from shell to python" scripts/scos-ipconfig.in
[feature 7fbd096] scos-ipconfig: rewrite scos-ipconfig scripts from shell to python
1 file changed, 167 insertions(+), 249 deletions(-)
```

FIGURE 4.49 – Git commit

- *Git push* : Utilisé pour charger le contenu d'un dépôt local vers un dépôt distant. Push nous permet de pousser les validations de notre dépôt local vers un dépôt distant.


```
~/scos-manuf-tools$ git push g701769 -f feature
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
```

FIGURE 4.50 – Git push

- *Ouvrez une marge request* : Accéder à notre plate-forme de dépôt distant (GitLab) et recherchez l’option permettant de créer une nouvelle étendue de requêtes. Sélectionner la succursale que nous venons de déplacer.

- *Décrivez notre marge request* : Saisir un titre et une description détaillée de notre demande de marge. Mentionner tout changement important que nous avons apporté.

Code Revue :

Les revues de code sont une étape essentielle dans le processus de développement collaboratif

- *Demandez une revue* : Si vous travaillez en équipe, demander à un ou plusieurs collaborateurs de passer en revue notre code.

- *Répondre aux commentaires* : les réviseurs peuvent commenter directement le code dans une marge request. Nous étudierons leurs suggestions et répondrons à leurs commentaires.

- *Effectuer les modifications si nécessaires* : En fonction des retours de la revue, nous devons peut-être effectuer des changements. Ajouter et committer ces modifications sur la même branche

4.6.3 Défis liés à la conversion de scripts shell en Python :

L’un des plus grands défis lors de la conversion de scripts Shell vers Python est la différence de syntaxe. Les scripts Shell sont écrits à l’aide d’une syntaxe spécifique au langage et Python possède sa propre syntaxe unique. Cela signifie que les développeurs doivent apprendre une nouvelle syntaxe pour écrire des scripts en Python. Un autre problème est la fonctionnalité. Les scripts Shell ont leurs propres fonctionnalités, telles que l’édition de fichiers et de dossiers. Ces fonctions peuvent ne pas être disponibles en Python ou avoir une syntaxe différente. Par conséquent, les développeurs doivent comprendre comment implémenter ces fonctionnalités dans Python afin de les utiliser efficacement.

4.7 Conclusion

Au cours de ce chapitre nous avons présenté les différentes étapes qui décrivent le bon fonctionnement de notre travail. C’est une description détaillée de la réalisation pratique du projet ainsi que l’environnement logiciel et matériel utilisé.

Conclusion Générale

La conversion de scripts shell en Python offre de nombreux avantages aux développeurs. Cela améliore non seulement l'efficacité et la productivité mais rend également les scripts plus lisibles maintenables et portables.

La société Sagemcom nous a fourni l'occasion pour réaliser notre projet d'été afin de convertir les scripts shell vers python .

Nous avons commencé notre projet par une étude de l'existant grâce à laquelle nous avons déterminé les besoins du projet et sélectionné la meilleure solution à utiliser

Ainsi , nous avons étudié la partie théorique de Software et hardware utilisé suivi par des outils nécessaires au projet .

Ensuite, nous avons présenté la partie conception qui décrit la méthodologie de notre projet.

Nous avons par la suite détaillé la réalisation de conversion à savoir Installer les outils nécessaires pour le projet. Ainsi, la test et conversion de script de shell à python .

Mais ,lors de la conversion d'un shell vers Python, il est important d'être conscient que la conversion peut introduire des défis, tels que les Différences de syntaxe et de fonctionnalité entre les deux langages. Par conséquent, avant de commencer la conversion, il est important de prendre le temps d'apprendre Python et de comprendre en quoi il diffère des scripts shell.

Bibliographie

<https://www.jedha.co/blog/git-et-github-definitions-differences-utilite>

https://docs.gitlab.com/ee/topics/manage_code.html

https://linuxhint.com/install_tftp_server_ubuntu/

<https://pylint.pycqa.org/en/latest/index.html>

<https://www.windriver.com/solutions/learning/embedded-linux>

<https://www.dir-tech.com/quelle-est-la-difference-entre-ftp-et-tftp/>

<https://www.ionos.fr/digitalguide/serveur/know-how/tftp/>

http://www.tcpipguide.com/free/t_TFTPGeneralOperationConnectionEstablishmentandClie.htm

<https://slidetodoc.com/ftp-file-transfer-protocol-tftp-trivial-ftp-cisc-2/>

<https://www.cadremploi.fr/editorial/conseils/conseils-carriere/detail/article/jose-pas-demander-mais-quest-ce-que-jira.html>

Résumé

Le stage est une activité pédagogique fondamentale de prise de contact avec le milieu professionnel. Car, pendant cette période on apprend à s'adapter profondément à la fonction future et recueillir les fruits de formation. Dans ce rapport j'ai rapporté l'essentiel de ce que j'ai effectué durant la période de stage.

J'ai commencé mon projet par une présentation générale de la société ainsi qu'une description du contexte général du projet. Ensuite j'ai présenté, une vue théorique sur le software de Sagemcom, matérielle et Implementation sur extender

En outre, j'ai détaillé la partie conception qui décrit la méthodologie de mon projet.

Enfin, j'ai analysé mon projet à l'aide de capture d'écran sur l'intitulé " Réécriture des outils du soft de test de Sagemcom depuis langage shell vers python".

Le but de mon travail est d'éviter les fuites de mémoire, de simplifier la syntaxe et de la rendre plus facile à comprendre.